

Angelos Michalopoulos

ITC 4568 – MACHINE LEARNING

April 30th, 2026

Final Project Report

Part 1 – Preprocessing the Data

1A

The first task of the final project was to write a python code that reads files from a user specified directory in order to create a single dataset based on those sensor csv files. The code can read two different folders, one for sensor 1 and one for sensor 2 and create a single dataset for each separately. The format of the single dataset created is the following:

<timestamp>,<value>,<label>

In order to achieve this format, the code had to collect the label for each value. The label was based on the state of the taps which was found in the title of the csv files that are currently read.

```
sensor_match_1 = re.search(r'Sensor([12])', 'data/ITC4568  
Final_Project_data/Sensor1_Leakage_2022-12-09_10-45-36')  
sensor_id = sensor_match_1.group(1)  
output_name = f"ALL_DATA_{sensor_id}.CSV"
```

The figure above depicts the code that creates the name for the dataset, in this case to create the file name for the first sensor, where it searches under the directory for the final Sensor1_Leakage and stores “Sensor1” to a variable named sensor_match_1. Following, it collects the sensor number in the sensor id variable and creates the correct output name for the file. The same process is followed again for the second sensor.

```

all_data_1 = []
[3]

for root, dirs, files in os.walk('data/ITC4568 Final_Project_data/Sensor1_Leakage_2022-12-09_10-45-36'):
    for file in files:
        if file.endswith(".csv"):
            root_folder = os.path.basename(root)
            parts = root_folder.split('_')
            if len(parts) >= 2:
                option = parts[1]
                if option == 'AllTapsClosed':
                    label = 0
                elif option == 'Tap10pen':
                    label = 1
                elif option == 'Tap20pen':
                    label = 2
                elif option == 'Tap30pen':
                    label = 3
                elif option == 'Tap40pen':
                    label = 4
                if label is not None:
                    file_path = os.path.join(root, file)
                    df = pd.read_csv(file_path)
                    df['label'] = label
                    all_data_1.append(df)
            if not all_data_1:
                print("No data found")
[4]

final_df_1 = pd.concat(all_data_1)
[5]

```

```

final_df_1.sort_values(by='Time', inplace=True)

```

[6]

```

final_df_1.to_csv(output_name, index=False)

```

```

print('success created: ', output_name)

```

[7]

```

success created: ALL_DATA_1.CSV

```

The screenshots above depict the creation of the actual file. First a list is created called `all_data_1` and the following code is an `os` for loop where it stores the root (the root file name), the dirs (the sub folder file names) and the files (the file names in the root folder). This loop reads the files and performs the internal for loop where for each file that ends with “.csv” it gets the root folder and starts the if statement processes to collect the labels of each value. Each subfile is responsible for each label and therefore it collects the label type by splitting the filename on the “_” and then passing the second part through the if statements. Based on which label they have a new df is created that also includes the label. After this loop is repeated for all the csv files for sensor-1 they

are stored on the list created previously and lastly, they are connected and saved under one data frame. The code then sorts the new data frame based on the time step so that the correct order is stored, and the time is always increasing. The final data frame is saved to a csv file under the name ALL_DATA_1.csv. The same code and logic is repeated for sensor-2 and saved under the appropriate name.

1B

The following task that had to be completed for part 1 of the final project was creating a new aggregate data file containing average values in intervals of 0.1 seconds for both sensor 1 and 2. The new aggregate data file contains the following features

<ts>,<average_value>,<min_value>,<max_value>,<label>

```
df1 = pd.read_csv('ALL_DATA_1.csv')  
[14]
```

```
df1['ts'] = (df1['Time']//0.1)*0.1  
df1['ts'] = df1['ts'].round(1)  
[15]
```

```
df1['group'] = (df1['label'] != df1['label'].shift()).cumsum()  
[16]
```

```
agg1 = df1.groupby(['ts', 'group', 'label']).agg(  
    average_value=('Modulus', 'mean'),  
    min_value=('Modulus', 'min'),  
    max_value=('Modulus', 'max')  
) .reset_index()  
[17]
```

```
final_df1 = agg1[['ts', 'average_value', 'min_value', 'max_value', 'label']]  
final_df1.to_csv('AGGR_DATA_1.csv', index=False)  
[18]
```

The screenshots above show how that task is completed. First the code stores the previously created raw data file as a pandas data frame and then creates the correct time intervals as the project demanded (0.1 seconds between each row). This is performed by dividing the timestamp by 0.1 and rounding it to the lowest nearest integer, for example if the time stamp said 0.34 it would be turned to 3.4 and then rounded down to 3. To turn the timestamp to 0.1 intervals the new timespan was then multiplied by 0.1, so in the previous example 3 would become 0.3. The time stamp is then rounded to make sure that the final results have only one important variable and do not drag. The second screenshot shows how the aggregate values are created. First, in order to make sure that the values are created for the same time period and the same label, the code created a group for each time the label is the same for rows. This is performed by shifting the label in order for the previous rows to know the following data points label. If they are the same, they are stored in the same group, if the label and the shifted label do not match then the code creates a new group for the different label. The new feature [group] is created and inserted to the data frame of the raw data. In order to create the aggregate values, which are the mean, the maximum and the minimum for each time bracket and group them, the code uses the groupby and the agg functions. The groupby calls the dataset based on the ts, group and label features and then uses the agg function to calculate and create the new features that the project demanded. For example, the groupby function collects all the values that have (ts=0.0, group=1, label=0) and with this grouping it calculates the mean, finds the maximum and the minimum and creates the new features for the new dataset. The final lines of code of this task create a new data frame based on this grouping and only stores the timestamp, average_value, min_value, max_value and the label. The code then stores it to a new csv file named AGGR_DATA_1.csv and the same is performed for the second sensor dataset which created the AGGR_DATA_2.csv file.

1C

The final task of part 1 of the final project dealt with merging the two-sensor based aggregate files to create one collective dataset that will be used to train the models that appear in the following two parts of the project. The first step in completing this task was to create new data frames using the two aggregated datasets.

```
agg_df1 = pd.read_csv('AGGR_DATA_1.csv')
agg_df2 = pd.read_csv('AGGR_DATA_2.csv')

agg_df1 = agg_df1.rename(columns={
    'average_value': 'sensor1_avg_val',
    'min_value': 'sensor1_min_val',
    'max_value': 'sensor1_max_val'
})

agg_df2 = agg_df2.rename(columns={
    'average_value': 'sensor2_avg_val',
    'min_value': 'sensor2_min_val',
    'max_value': 'sensor2_max_val'
})
```

Their features were also renamed to make it clear which features belong to sensor 1 and which belong to sensor 2.

```

merged_df = pd.merge(agg_df1, agg_df2, on=['ts', 'label'], how='outer')
[25]

merged_df.sort_values(by='ts', inplace=True)
[26]

columns = ['ts',
           'sensor1_avg_val', 'sensor1_min_val', 'sensor1_max_val',
           'sensor2_avg_val', 'sensor2_min_val', 'sensor2_max_val',
           'label']
[27]

merged_df = merged_df[columns]
[28]

change_columns = ['sensor1_avg_val', 'sensor1_min_val', 'sensor1_max_val',
                  'sensor2_avg_val', 'sensor2_min_val', 'sensor2_max_val']
[29]

merged_df[change_columns] = merged_df[change_columns].astype(object)
[30]

merged_df.fillna('?', inplace=True)
[31]

merged_df.to_csv('AGGR_DATA_ALL.csv', index=False)
[32]

```

The above screenshot shows the merging process for the two files. The merging is performed with the `pd.merge` command where the two aggregated datasets are merged based on the timestamp and label features. The merge type of `outer` was chosen in the cases where values from both the sensors do not match on the timestamp or the label and thus are left blank for one of the sensors. The new merged data frame is then sorted by the time stamp so that it is always increasing. The project then demanded that the values that are missing be replaced with the symbol “?”. In order to do this, the features first had to be turned into objects. Without this procedure the code gave an error that the symbol cannot be inputted since it is not a float number. To complete this process, the features that had to be changed had to be selected, every feature except the timestamp and the label. Finally, after the change is made with the command `.fillna(“?”, inplace=True)` the null points have the symbol that is requested by the project. The final merged dataset is saved in a .csv file called `AGGR_DATA_ALL.csv`.

PART 2 – Training ML Models on the Data

2A

The second part of the final project dealt with the creation of the base machine learning models to train with the merged dataset created in the third task of part 1. The machine learning models used for this part are a neural network using the sklearn library and a decision tree model.

The first task that has to be performed in order to be able to train the models is to clean the dataset. This entails dealing with the rows that have the “?” symbol for when data is missing.

```
finaldf.shape
```

```
[35]
```

```
(8183, 8)
```

```
(finaldf == '?').sum()
```

```
[36]
```

ts	0
sensor1_avg_val	28
sensor1_min_val	28
sensor1_max_val	28
sensor2_avg_val	49
sensor2_min_val	49
sensor2_max_val	49
label	0
dtype:	int64

The code from the screenshot above shows the shape of the merged data and then the number of instances where there exists the symbol “?”. From the code already shown so far, we know that for each feature that belongs to sensor 1 and sensor 2 that it is the same 28 rows and 49 rows missing which amounts to 77 rows where there are missing values. From the shape we can see that the dataset has 8,183 rows and 77 rows amount for around 1 present of those rows. Since that is a small number, we can drop the missing rows and have very minimal loss of information. To complete the above, the rows with the symbol “?” have to be replaced with an np.nan and then the rows can be dropped.

```
finaldf = finaldf.replace('?', np.nan)
```

```
cleandf = finaldf.dropna()
```

The following task concerns removing the timestamp feature and then splitting the dataset into train/validation/test datasets with a 40-30-30 split.

```
x = cleandf.drop(columns=['ts', 'label'])  
y = cleandf['label']
```

The code above depicts the preparation of the split as well as dropping the ts feature. The x dataset contains every feature with exception of the timestamp and the target (label) feature, and the y dataset is just the target.

After creating the x dataset and the y dataset, the following objective was to complete the train/validation/test splits, starting with creating the 40% train set using the train_test_split with the arguments x, y, train_size=0.4, random_state=42, stratify=y. More specifically, train_size=0.4 does the split so that the train set gets 40% of the available dataset and the remaining 60% goes to the val_test sets (both x and y). Stratify means keeping the proportionality for each target for the splits. Afterwards, one more split is also done to split the x_val_test and y_val_test to validation and test splits and, since the assignment demands an overall 40-30-30 split, the validation and test split is with test_size 0.5. To follow correct practices the data was also scaled which removed the different order of magnitude.

```
x_train = scaler.fit_transform(x_train)
x_validation = scaler.transform(x_validation)
x_test = scaler.transform(x_test)
```

Now that the preprocessing of the data is completed the model training can proceed. The first step was training the neural network with the topology that the assignment asks for. For this task the sklearn library is used which means that the activation function for the output layer as well as the loss function are being handled by the library itself. The sklearn MLPClassifier uses the SoftMax function when it deals with multi-class classification problems and the loss function used is Cross-Entropy loss function. The project demands that the topology of the NN is as follows: 10 hidden nodes, 5 output nodes (one for each class) and the hidden node activation function is ReLU.

```
nn = MLPClassifier(hidden_layer_sizes=(10,), activation='relu', solver='adam', max_iter=1000, random_state=42)
```

After creating the model and fitting it to the data using the .fit() function the results, based on the accuracy and the F1 metric, are the following.

accuracy: 0.40542763157894735

F1 score: 0.29602877727847166

The accuracy metric shows the percentage of predictions that were right for the test set which is unseen data. The results show that the model has a prediction of around 41% which is very low and shows that the model gets the prediction wrong the majority of the time. The F1 score is the harmonic mean of two other metrics, recall and precision, which both give more insight on the quality of the predictions. An F1 score of around 0.30 is also very low which means that not only a lot of true positives are missed but also many of the predictions made do not belong to the correct class.

After the neural network, the next model that had to be trained was a decision tree model, but before the training could proceed the project asked for the test and the validation datasets to be concatenated. The np.concatenate([y_validation, y_test], axis=0) function was used to achieve it. Using the correct datasets, the decision tree model produced these final results.

accuracy: 0.2865953947368421

F1: 0.28606246755895337

From the results we can tell that the overall accuracy is worse than the neural networks accuracy and much closer to random guessing, which for a dataset with five classes would have an accuracy around 20%. On the other hand, from the F1 score we can infer that the results are in reality a lot closer than they seem. The NN F1 score and the DT F1 score are nearly identical which suggests that while the DT model predicted less correct predictions overall the quality of those predictions are about the same as with the neural network model.

PART 3 – Improving the Results

3A

The third part of the project dealt with improving the results that we got from the baseline models. The first way to achieve this is by performing feature engineering and adding new features to the merged dataset, which should in theory help with giving the models more data patterns to learn from. In order to complete this task with the merged data, the steps from part 1 were repeated and a new merged dataset was created that also featured the new data features. The features that were added were the following. First added feature was the standard deviation for each time bracket. This feature was created using the groupby and agg functions same as the mean and the max values that were present in the original merged data.

```

df1 = pd.read_csv('ALL_DATA_1.csv')
df1['ts'] = (df1['Time']/0.1)*0.1
df1['ts'] = df1['ts'].round(1)
df1['group'] = (df1['label'] != df1['label'].shift()).cumsum()
agg1 = df1.groupby(['ts', 'group', 'label']).agg(
    sensor_1_average_value=('Modulus', 'mean'),
    sensor_1_min_value=('Modulus', 'min'),
    sensor_1_max_value=('Modulus', 'max'),
    sensor_1_std_value=('Modulus', 'std')
).reset_index()

agg1 = agg1.sort_values('ts').reset_index(drop=True)

```

The following feature was the one for the absolute max value over the last 10 seconds which was done using the function

```
agg1['sensor_1_max_abs_10s']=agg1['sensor_1_average_value'].abs().rolling(100,min_periods=1).max()
```

and, the final feature that was added in order to improve the final results for both models was the absolute minim over the last 10 seconds which was once again done using a similar line of code, specifically

```
agg1['sensor_1_min_abs_10s']=agg1['sensor_1_average_value'].abs().rolling(100,min_periods=1).min() .
```

Afterwards, the models that were created and trained with the newly created merged dataset which included the three new features. The data was also once again split using the same splitting ratios of 40-30-30.

The results of the neural network are as follow:

accuracy: 0.6747532894736842

F1 score: 0.6683264744385797

The results show that the added features help the model very much. The final accuracy of the model increased by nearly 26% (41%→67%) which is a very good increase and now the model shows that it also predicts the majority of the data correctly. The most important increase with the new data was the F1 score which more than doubled compared to the original results. This shows that along with the overall accuracy of the predictions their quality also increased and shows that the model now also recognizes more of the true positives of the data and has also lowered the rate of false positives.

When it comes to the decision tree, we see that the model performs nearly perfectly with the new data.

accuracy: 0.9701891447368421

f1: 0.9701897255084229

From the above results we see that not only did the DT model surpass the NN model, which for the two base lines the DT model was worse, in overall accuracy by a lot reaching nearly perfect accuracy but that the F1 score also matches the accuracy and performs almost perfectly, which is once again much better than the NN model. It should be mentioned though that when the prediction accuracy and F1 are this good it can be a case of either data leakage or overfitting. Since those results appear with the unseen data we can be a little surer of the results but we should be cautious.

3B

The final objective of the final project also dealt with improving the results further but by changing the Neural network topologies and activation functions not just the data. This part also asks to test a convolutional neural network on the dataset.

The first change that was tested for the NN model was twofold, by increasing the number of hidden layers and the hidden nodes in the first layer.

```
nn_part3b = MLPClassifier(hidden_layer_sizes=(64,32), activation='relu', solver='adam', max_iter=1000,
    random_state=42)
nn_part3b.fit(x_train, y_train)
```

The new hidden layers have 64 hidden nodes on the first layer and 32 hidden layers on the second hidden layer. The activation function is kept the same for this test.

accuracy: 0.8408717105263158

F1 score: 0.8410475585937526

The results above show that with this new topology the NN model performs much better than both the base line model and the model with just the new data features but the same topology. Compared to the original NN model the accuracy has more than doubled in performance (41% to 84%). Furthermore, the F1 score has also increased and now performs as well as the overall accuracy which tells us that the predictions of the model are now more balanced instead of just guessing one class more than the others, which was one of the reasons that the base model had such a big disparity from accuracy and F1 score.

The next change in topology had to do with changing the activation function. Keeping the hidden layers the same with the previous change, ReLU was changed to tanh and the results once again increased to show an even more reliable model.

accuracy: 0.9009046052631579

F1 score: 0.900436646333325

The final changed topology was adding yet one more layer of hidden nodes as well as increasing the number of hidden nodes for each layer from (64, 32) to (256, 100, 50) but keeping the activation function tanh.

```
nn_part3b = MLPClassifier(hidden_layer_sizes=(256, 100, 50), activation='tanh', solver='adam', max_iter=1000,
    random_state=42)
nn_part3b.fit(x_train, y_train)
y_pred = nn_part3b.predict(x_test)
y_true = y_test
print("accuracy for hidden_layer_sizes=(256, 100, 50), activation='tanh': ", accuracy_score(y_true, y_pred))
print("F1 score for hidden_layer_sizes=(256, 100, 50), activation='tanh': ", f1_score(y_true, y_pred,
    average='weighted'))
```

With this topology and activation function the model performs the best overall producing these results.

accuracy: 0.9313322368421053

F1 score: 0.9310385255672706

With this changed topology the accuracy and the F1 reach a percentage of near the mid-90s and show that not only is the model useful by predicting almost all correctly overall, but that the model is also trustworthy because the predictions are balanced, and all the classes are correctly predicted and represented.

Finally for the final project, a convolutional neural network had to be trained and evaluated based on the merged dataset as well the accuracy and F1 score. First, the TensorFlow library had to be imported but in this attempt python kept crashing and not loading the library, so instead of loading TensorFlow keras was attempted. However, the only way to achieve this is by first loading the os library and forcing the keras backend to be torch by using the pyTorch instead of the TensorFlow backend, as seen in the screenshot below.

```
import os
os.environ["KERAS_BACKEND"] = "torch"
import keras
```

Now the library loaded normally, and the creation of the CNN model proceeded.

```
model = keras.Sequential([
    keras.layers.Input((12, 1)),
    keras.layers.Conv1D(32, 3, activation='relu', padding='same'),
    keras.layers.Conv1D(64, 3, activation='relu', padding='same'),
    keras.layers.GlobalAveragePooling1D(),
    keras.layers.Dense(5, activation='softmax'),
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)
res = model.evaluate(x_test, y_test)
print('test loss, acc=', res)
y_pred_cnn = model.predict(x_test).argmax(axis=1)
print("accuracy for CNN: ", accuracy_score(y_test, y_pred_cnn))
print("F1 score for CNN: ", f1_score(y_test, y_pred_cnn, average='weighted'))
```

From the screen shot above we can see that the model builds the network using the .sequential function. The input layer is declared to have a shape of 12 positions along an axis with 1 channel and so the model expects that the shape of the input will be batches of this shape. The first convolutional layer (.Conv1D()) learns 32 filters, each looking at a window of 3 consecutive

positions at a time. As each filter slides across the 12-position input, it computes a weighted sum of the 3 values it sees and applies ReLU padding is included so that the output stays the same length as the input. For the second convolutional layer, the same idea was performed but with 64 filters instead of the previous layer's 32-channel output. The next function is the `GlobalAveragePooling1D()` function which collapses the 12 positions of the axis by averaging all of them for each of the 64 channels from the second convolutional layer. Finally, the dense function creates a fully connected output layer with 5 output nodes and an activation function of SoftMax.

After creating the final model it was fitted and trained to the dataset and produced these results:
accuracy: 0.4420230263157895

F1 score: 0.3330472198222956

From these results we can see that for the CNN there is almost no improvement from the baseline model and that may mean that the added complexity does not help with the data that was used.

Conclusion

This final project shows the full machine learning pipeline, from preprocessing raw sensor data to training and comparing multiple models. The baseline neural network and decision tree both performed poorly on the merged dataset (41% and 29% accuracy respectively), only slightly better than just random guessing. Feature engineering had the largest single impact which by adding the standard deviation and the rolling 10-second max-absolute and min-absolute values raised the neural network to 67% and the decision tree to 97% accuracy. Testing different neural network topologies and activation functions improved the model even more, with the (256, 100, 50) tanh architecture reaching 93% accuracy. The convolutional neural network, however, did not improve over the baseline model (44% accuracy), as the CNN architecture did not match the tabular structure of the input features.

Appendix

Part 1

```
"""**reading from directory and creating all_data files**"""

sensor_match_1 = re.search(r'Sensor([12])', 'data/ITC4568
Final_Project_data/Sensor1_Leakage_2022-12-09_10-45-36')
sensor_id = sensor_match_1.group(1)
output_name = f"ALL_DATA_{sensor_id}.CSV"

all_data_1 = []

for root, dirs, files in os.walk('data/ITC4568
Final_Project_data/Sensor1_Leakage_2022-12-09_10-45-36'):
    for file in files:
        if file.endswith(".csv"):
            root_folder = os.path.basename(root)
            parts = root_folder.split('_')
            if len(parts) >= 2:
                option = parts[1]
                if option == 'AllTapsClosed':
                    label = 0
                elif option == 'Tap1Open':
                    label = 1
                elif option == 'Tap2Open':
                    label = 2
                elif option == 'Tap3Open':
                    label = 3
                elif option == 'Tap4Open':
                    label = 4
                if label is not None:
                    file_path = os.path.join(root, file)
                    df = pd.read_csv(file_path)
                    df['label'] = label
                    all_data_1.append(df)

if not all_data_1:
    print("No data found")

final_df_1 = pd.concat(all_data_1)

final_df_1.sort_values(by='Time', inplace=True)

final_df_1.to_csv(output_name, index=False)
print('success created: ', output_name)

sensor_match_2 = re.search(r'Sensor([12])', 'data/ITC4568
Final_Project_data/Sensor2_Leakage_2022-12-09_10-46-58')
sensor_id = sensor_match_2.group(1)
output_name = f"ALL_DATA_{sensor_id}.CSV"

all_data_2 = []

for root, dirs, files in os.walk('data/ITC4568
Final_Project_data/Sensor2_Leakage_2022-12-09_10-46-58'):
    for file in files:
```

```

        if file.endswith(".csv"):
            root_folder = os.path.basename(root)
            parts = root_folder.split('_')
            if len(parts) >= 2:
                option = parts[1]
                if option == 'AllTapsClosed':
                    label = 0
                elif option == 'Tap1Open':
                    label = 1
                elif option == 'Tap2Open':
                    label = 2
                elif option == 'Tap3Open':
                    label = 3
                elif option == 'Tap4Open':
                    label = 4
                if label is not None:
                    file_path = os.path.join(root, file)
                    df = pd.read_csv(file_path)
                    df['label'] = label
                    all_data_2.append(df)

if not all_data_2:
    print("No data found")

final_df_2 = pd.concat(all_data_2)

final_df_2.sort_values(by='Time', inplace=True)

final_df_2.to_csv(output_name, index=False)
print('success created: ', output_name)

"""**creating aggregated files for data 1 and data 2**"""

df1 = pd.read_csv('ALL_DATA_1.csv')

df1['ts'] = (df1['Time']//0.1)*0.1
df1['ts'] = df1['ts'].round(1)

df1['group'] = (df1['label'] != df1['label'].shift()).cumsum()

agg1 = df1.groupby(['ts', 'group', 'label']).agg(
    average_value=('Modulus', 'mean'),
    min_value=('Modulus', 'min'),
    max_value=('Modulus', 'max')
).reset_index()

final_df1 = agg1[['ts', 'average_value', 'min_value', 'max_value', 'label']]
final_df1.to_csv('AGGR_DATA_1.csv', index=False)

df2 = pd.read_csv('ALL_DATA_2.csv')

df2['ts'] = (df2['Time']//0.1)*0.1
df2['ts'] = df2['ts'].round(1)

df2['group'] = (df2['label'] != df2['label'].shift()).cumsum()

agg2 = df2.groupby(['ts', 'group', 'label']).agg(
    average_value=('Modulus', 'mean'),

```

```

        min_value=('Modulus', 'min'),
        max_value=('Modulus', 'max')
    ).reset_index()

final_df2 = agg2[['ts', 'average_value', 'min_value', 'max_value', 'label']]
final_df2.to_csv('AGGR_DATA_2.csv', index=False)

"""**merging files**"""

agg_df1 = pd.read_csv('AGGR_DATA_1.csv')
agg_df2 = pd.read_csv('AGGR_DATA_2.csv')

agg_df1 = agg_df1.rename(columns={
    'average_value': 'sensor1_avg_val',
    'min_value': 'sensor1_min_val',
    'max_value': 'sensor1_max_val'
})

agg_df2 = agg_df2.rename(columns={
    'average_value': 'sensor2_avg_val',
    'min_value': 'sensor2_min_val',
    'max_value': 'sensor2_max_val'
})

merged_df = pd.merge(agg_df1, agg_df2, on=['ts', 'label'], how='outer')

merged_df.sort_values(by='ts', inplace=True)

columns = ['ts',
            'sensor1_avg_val', 'sensor1_min_val', 'sensor1_max_val',
            'sensor2_avg_val', 'sensor2_min_val', 'sensor2_max_val',
            'label']

merged_df = merged_df[columns]

change_columns = ['sensor1_avg_val', 'sensor1_min_val', 'sensor1_max_val',
                  'sensor2_avg_val', 'sensor2_min_val', 'sensor2_max_val']

merged_df[change_columns] = merged_df[change_columns].astype(object)

merged_df.fillna('?', inplace=True)

merged_df.to_csv('AGGR_DATA_ALL.csv', index=False)

```

Part 2

```

"""**Model creation and training**

**Part 2 A**
"""

finaldf = pd.read_csv('AGGR_DATA_ALL.csv')

finaldf.head()

```

```

finaldf.shape

(finaldf == '?').sum()

finaldf = finaldf.replace('?', np.nan)

finaldf.isnull().sum()

cleandf = finaldf.dropna()

cleandf.head()

cleandf.shape

x = cleandf.drop(columns=['ts', 'label'])
y = cleandf['label']

x.head()

y.head()

scaler = StandardScaler()

x_train, x_val_test, y_train, y_val_test = train_test_split(x, y,
train_size=0.4, random_state=42, stratify=y)

x_train.shape

x_validation, x_test, y_validation, y_test = train_test_split(x_val_test,
y_val_test, test_size=0.5, random_state=42, stratify=y_val_test)

x_test.shape

x_validation.shape

x_train = scaler.fit_transform(x_train)
x_validation = scaler.transform(x_validation)
x_test = scaler.transform(x_test)

nn = MLPClassifier(hidden_layer_sizes=(10,), activation='ReLU',
solver='adam', max_iter=1000, random_state=42)

nn.fit(x_train, y_train)

print({nn.out_activation_})

y_pred = nn.predict(x_test)
y_true = y_test

print("accuracy for base neural network: ", accuracy_score(y_true, y_pred))

print("F1 score for base neural network: ", f1_score(y_true, y_pred,
average='weighted'))

"""**Part 2B**

```

```

"""

x_test_dt = np.concatenate([x_validation, x_test], axis=0)

y_test_dt = np.concatenate([y_validation, y_test], axis=0)

dt = DecisionTreeClassifier(random_state=42)
dt.fit(x_train, y_train)

y_pred_dt = dt.predict(x_test_dt)

print("accuracy for base decision tree: ", accuracy_score(y_test_dt,
y_pred_dt))

print("f1 for base decision tree: ", f1_score(y_test_dt, y_pred_dt,
average='weighted'))

```

Part 3

```

"""**Part 3**"""

df1 = pd.read_csv('ALL_DATA_1.csv')
df1['ts'] = (df1['Time']/0.1)*0.1
df1['ts'] = df1['ts'].round(1)
df1['group'] = (df1['label'] != df1['label'].shift()).cumsum()
agg1 = df1.groupby(['ts', 'group', 'label']).agg(
    sensor_1_average_value=('Modulus', 'mean'),
    sensor_1_min_value=('Modulus', 'min'),
    sensor_1_max_value=('Modulus', 'max'),
    sensor_1_std_value=('Modulus', 'std')
).reset_index()

agg1 = agg1.sort_values('ts').reset_index(drop=True)

agg1['sensor_1_max_abs_10s'] =
agg1['sensor_1_average_value'].abs().rolling(100, min_periods=1).max()
agg1['sensor_1_min_abs_10s'] =
agg1['sensor_1_average_value'].abs().rolling(100, min_periods=1).min()

final_df1 = agg1[['ts', 'sensor_1_average_value', 'sensor_1_min_value',
'sensor_1_max_value', 'sensor_1_std_value',
'sensor_1_max_abs_10s', 'sensor_1_min_abs_10s', 'label']]
final_df1.to_csv('AGGR_DATA_1_part3.csv', index=False)

df2 = pd.read_csv('ALL_DATA_2.csv')
df2['ts'] = (df2['Time']/0.1)*0.1
df2['ts'] = df2['ts'].round(1)
df2['group'] = (df2['label'] != df2['label'].shift()).cumsum()
agg2 = df2.groupby(['ts', 'group', 'label']).agg(
    sensor_2_average_value=('Modulus', 'mean'),
    sensor_2_min_value=('Modulus', 'min'),
    sensor_2_max_value=('Modulus', 'max'),
    sensor_2_std_value=('Modulus', 'std')
).reset_index()

```

```

agg2 = agg2.sort_values('ts').reset_index(drop=True)

agg2['sensor_2_max_abs_10s'] =
agg2['sensor_2_average_value'].abs().rolling(100, min_periods=1).max()
agg2['sensor_2_min_abs_10s'] =
agg2['sensor_2_average_value'].abs().rolling(100, min_periods=1).min()
final_df2 = agg2[['ts', 'sensor_2_average_value', 'sensor_2_min_value',
'sensor_2_max_value', 'sensor_2_std_value', 'sensor_2_max_abs_10s',
'sensor_2_min_abs_10s', 'label']]
final_df2.to_csv('AGGR_DATA_2_part3.csv', index=False)

agg_df1 = pd.read_csv('AGGR_DATA_1_part3.csv')
agg_df2 = pd.read_csv('AGGR_DATA_2_part3.csv')

merged_df = pd.merge(agg_df1, agg_df2, on=['ts', 'label'], how='outer')
merged_df.sort_values(by='ts', inplace=True)

merged_df.head()

columns = ['ts', 'sensor_1_average_value', 'sensor_1_min_value',
'sensor_1_max_value', 'sensor_1_std_value', 'sensor_1_max_abs_10s',
'sensor_1_min_abs_10s', 'sensor_2_average_value', 'sensor_2_min_value',
'sensor_2_max_value', 'sensor_2_std_value',
'sensor_2_max_abs_10s', 'sensor_2_min_abs_10s', 'label']

merged_df = merged_df[columns]

merged_df.head()

change_columns = ['sensor_1_average_value', 'sensor_1_min_value',
'sensor_1_max_value', 'sensor_1_std_value', 'sensor_1_max_abs_10s',
'sensor_1_min_abs_10s', 'sensor_2_average_value', 'sensor_2_min_value',
'sensor_2_max_value', 'sensor_2_std_value',
'sensor_2_max_abs_10s', 'sensor_2_min_abs_10s']
merged_df[change_columns] = merged_df[change_columns].astype(object)

merged_df.fillna('?', inplace=True)

merged_df.to_csv('AGGR_DATA_ALL_part3.csv', index=False)

finaldf_part3 = pd.read_csv('AGGR_DATA_ALL_part3.csv')
finaldf_part3.head()

finaldf_part3.shape

(finaldf_part3 == '?').sum()

finaldf_part3 = finaldf_part3.replace('?', np.nan)

finaldf_part3.isnull().sum()

cleandf_part3 = finaldf_part3.dropna()

cleandf_part3.head()

```

```

cleandf_part3.shape

x = cleandf_part3.drop(columns=['ts', 'label'])
y = cleandf_part3['label']
x.head()
y.head()
scaler = StandardScaler()
x_train, x_val_test, y_train, y_val_test = train_test_split(x, y,
train_size=0.4, random_state=42, stratify=y)

x_train.shape

x_validation, x_test, y_validation, y_test = train_test_split(x_val_test,
y_val_test, test_size=0.5, random_state=42, stratify=y_val_test)

x_test.shape

x_validation.shape

x_train = scaler.fit_transform(x_train)
x_validation = scaler.transform(x_validation)
x_test = scaler.transform(x_test)

nn_part3 = MLPClassifier(hidden_layer_sizes=(10,), activation='ReLU',
solver='adam', max_iter=1000, random_state=42)
nn_part3.fit(x_train, y_train)

print({nn_part3.out_activation_})

y_pred = nn_part3.predict(x_test)
y_true = y_test

print("accuracy for hidden_layer_sizes=(10,), activation='ReLU' with new data
features: ", accuracy_score(y_true, y_pred))
print("F1 score for hidden_layer_sizes=(10,), activation='ReLU' with new data
features: ", f1_score(y_true, y_pred, average='weighted'))

x_test_dt_part3 = np.concatenate([x_validation, x_test], axis=0)
y_test_dt_part3 = np.concatenate([y_validation, y_test], axis=0)
dt = DecisionTreeClassifier(random_state=42)
dt.fit(x_train, y_train)
y_pred_dt = dt.predict(x_test_dt_part3)
print("accuracy for decision tree with new data features: ",
accuracy_score(y_test_dt_part3, y_pred_dt))
print("f1 for decision tree with new data features: ",
f1_score(y_test_dt_part3, y_pred_dt, average='weighted'))

"""**Part 3B1 (different topologies)**"""

nn_part3b = MLPClassifier(hidden_layer_sizes=(64,32), activation='ReLU',
solver='adam', max_iter=1000, random_state=42)
nn_part3b.fit(x_train, y_train)

y_pred = nn_part3b.predict(x_test)
y_true = y_test

```

```

print("accuracy for hidden_layer_sizes=(64,32), activation='ReLU': ",
accuracy_score(y_true, y_pred))
print("F1 score for hidden_layer_sizes=(64,32), activation='ReLU': ",
f1_score(y_true, y_pred, average='weighted'))

nn_part3b = MLPClassifier(hidden_layer_sizes=(64, 32), activation='tanh',
solver='adam', max_iter=1000, random_state=42)
nn_part3b.fit(x_train, y_train)
y_pred = nn_part3b.predict(x_test)
y_true = y_test
print("accuracy for hidden_layer_sizes=(64, 32), activation='tanh': ",
accuracy_score(y_true, y_pred))
print("F1 score for hidden_layer_sizes=(64, 32), activation='tanh': ",
f1_score(y_true, y_pred, average='weighted'))

nn_part3b = MLPClassifier(hidden_layer_sizes=(256, 100, 50),
activation='tanh', solver='adam', max_iter=1000, random_state=42)
nn_part3b.fit(x_train, y_train)
y_pred = nn_part3b.predict(x_test)
y_true = y_test
print("accuracy for hidden_layer_sizes=(256, 100, 50), activation='tanh': ",
accuracy_score(y_true, y_pred))
print("F1 score for hidden_layer_sizes=(256, 100, 50), activation='tanh': ",
f1_score(y_true, y_pred, average='weighted'))

"""**3B2 (CNN) **"""

import os
os.environ["KERAS_BACKEND"] = "torch"
import keras

model = keras.Sequential([
    keras.layers.Input((12, 1)),
    keras.layers.Conv1D(32, 3, activation='ReLU', padding='same'),
    keras.layers.Conv1D(64, 3, activation='ReLU', padding='same'),
    keras.layers.GlobalAveragePooling1D(),
    keras.layers.Dense(5, activation='softmax'),
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)
res = model.evaluate(x_test, y_test)
print('test loss, acc=', res)
y_pred_cnn = model.predict(x_test).argmax(axis=1)
print("accuracy for CNN: ", accuracy_score(y_test, y_pred_cnn))
print("F1 score for CNN: ", f1_score(y_test, y_pred_cnn, average='weighted'))

```

requirements.txt:

```

pandas==2.3.1
numpy==2.0.2
scikit-learn==1.6.1
keras==3.10.0
torch==2.8.0

```


